

NEW FORTRAN PROGRAMS FOR ANGULAR MOMENTUM COEFFICIENTS

K. SRINIVASA RAO *

Institut für Theoretische Kernphysik der Universität Bonn, Nussallee 16, D-53, Bonn, West Germany

and

K. VENKATESH

Department of Physics, University of Mysore, Mysore-570 006, India

Received 16 January 1976; in revised form 21 November 1977

PROGRAM SUMMARY

Title of program: ANGMOM

Catalogue number: ACYK

Program obtainable from: CPC Program Library, Queen's University of Belfast, N. Ireland (see application form in this issue)

Computer: IBM 370/168; *Installation:* University of Bonn Computation Centre

Operating system: OS-VS2

Programming language used: FORTRAN IV(G)

High speed store required: 14544 words

No. of bits in a word: 32 (4 bytes)

Overlay structure: none

No. of magnetic tapes required: none

Other peripherals used: card reader and line printer

No. of cards in combined program and test deck: 904

Card punching code: BCD

Keywords: general purpose, nuclear, atomic, molecular, angular momentum, nuclear structure, nuclear reactions,

matrix element, Clebsch–Gordan coefficient, Racah coefficient, high spin states, coupling and recoupling coefficients, shell model, generalized hypergeometric functions, rotation group

Nature of the physical problem

The program calculates the Clebsch–Gordan and Racah coefficients, using conventional formulae [1], the improved codes due to Wills [2] and Bretz [3] and in terms of generalized hypergeometric functions of unit argument defined by us [4,5]. The Clebsch–Gordan coefficient arises in the coupling of two angular momenta and the Racah coefficient arises in the recoupling of three angular momenta. These coefficients are of fundamental importance in nuclear, atomic and molecular physics calculations.

Method of solution

In refs. 4 and 5 we have shown that a set of six ${}_3F_2(1)$ s and two equivalent sets of ${}_4F_3(1)$ s are necessary and sufficient to account, respectively, for the known symmetries of the Clebsch–Gordan and Racah coefficients. Wills [2] has shown that the conventional approach detailed by Tamura [6], when used for large values of angular momenta, will lead, in single-precision, to inaccurate results due to the computer round-off error and he proposed an improved method which is more accurate and faster. The hypergeometric functions have this improvement naturally built-in and their use for the Clebsch–Gordan and Racah coefficients yields new FORTRAN programs which are shown here to yield the same accuracy as that attained by Wills [2] and Bretz [3] and are even faster.

Typical running time

The execution time required to calculate a Clebsch–Gordan or a Racah coefficient depends mainly upon the values of

* On leave of absence from: MATSCIENCE, The Institute of Mathematical Sciences, Madras-600 020, India.

its arguments. For small values of angular momenta, the conventional method of Tamura [6], the method of Wills [2] and Bretz [3] and our methods all yield the same results. But, for large values of angular momenta, in single-precision, while Tamura's method [6] breaks down due to large round-off errors, the Wills [2]–Bretz [3] method is not only more accurate but is approximately twice as fast as Tamura's method with negligibly small round-off errors. Our programs based on the use of generalized hypergeometric functions and their properties, are as accurate as those of Wills and Bretz and are even faster than their programmes by about 5–15%.

LONG WRITE-UP

1. Introduction

The present programs are FORTRAN functions named C, CF, CW, W, WF, W4F and WBZ. For a given set of data, the Clebsch–Gordan coefficient [1] for coupling of any two angular momenta is computed using the three routines C, CF and CW and for another given set of data, the Racah [1] coefficient for recoupling of three angular momenta is computed using the four routines W, WF, W4F and WBZ. The time taken for each method of computation has been found by the use of an interval timer routine implemented on IBM 370/168.

The need for Clebsch–Gordan and Racah coefficients usually arises in nuclear shell model [2] and nuclear reaction calculations, as well as in atomic and molecular physics calculations. The computation of matrix elements requires these basic angular momentum coefficients.

In coding the present function programs C, CF and CW for the Clebsch–Gordan coefficient, we make use of the conventional single sum expression given originally by Racah [1], as detailed by Tamura [3], the set of generalized hypergeometric functions of unit argument given by one of us [4] and the expression given by Wills [5]. In the coding of the function programs W, WF, W4F and WBZ for the Racah coefficient, we employ the conventional single sum series expression given originally by Racah [1], as detailed by Tamura [3], the set I of generalized hypergeometric functions of unit argument, ${}_4F_3(1)$ s, given by us [6], the equivalent set II of ${}_4F_3(1)$ s given by us [7] and the expression given by Bretz [8]. It is indeed surprising that the observation

References

- [1] G. Racah, Phys. Rev. 62 (1942) 438.
- [2] J.G. Wills, Comput. Phys. Commun. 2 (1971) 381.
- [3] V. Bretz, Acta Phys. Acad. Sci. Hungaricas 40 (1976) 255.
- [4] K. Srinivasa Rao, Proc. 6th Int. Coll. on Gp. Theor. Methods in Phys., Tübingen (1977) and to be published.
- [5] K. Srinivasa Rao, T.S. Santhanam and K. Venkatesh, Jour. Math. Phys. 16 (1975) 1528 and K. Srinivasa Rao and K. Venkatesh, Proc. of 5th Int. Coll. on Gp. Theor. Methods in Phys., Montreal (1976).
- [6] T. Tamura, Comput. Phys. Commun. 1 (1970) 337.

regarding the set of ${}_3F_2(1)$ s for the Clebsch–Gordan coefficient and the sets of ${}_4F_3(1)$ s for the Racah coefficient, their necessity and sufficiency to account for the known 72 and 144 symmetries, respectively, of these angular momentum coefficients, has not been made till now in the literature [9]. In our function programs CF, WF and W4F, based on the set of six ${}_3F_2(1)$ s, the set I of ${}_4F_3(1)$ s and the set II of ${}_4F_3(1)$ s, respectively, we adopt Horner's rule [10] for computing the generalized hypergeometric functions and exploit the simple property that when any one of the numerator parameters of a ${}_pF_q(1)$ is zero, then it has the value 1.

In section 2, we give the mathematical formulae used and the method of calculation. In section 3, we give a description of the program structure as well as its operation. Finally, in section 4, we discuss the results and the relative merits of the function programs.

2. Mathematical formulae

In this section, we write down all the expressions we use in our program for calculating the Clebsch–Gordan and Racah coefficients, in a unified notation.

The conventional expression for the Clebsch–Gordan coefficient [1] is given by:

$$C(j_1 j_2 j_3; m_1 m_2 m_3) = \delta(m_1 + m_2, m_3) [j_3] \Delta(j_1 j_2 j_3)$$

$$\prod_{i=1}^3 [(j_i + m)! (j_i - m_i)!]^{1/2}$$

$$\sum_t (-1)^t [t! \prod_{k=1}^2 (t - \alpha_k)! \prod_{l=1}^3 (\beta_l - t)!]^{-1}, \quad (2.1)$$

where

$$t_{\min} \leq t \leq t_{\max}, \quad t_{\min} = \max(0, \alpha_1, \alpha_2),$$

$$t_{\max} = \min(\beta_1, \beta_2, \beta_3), \quad \beta_1 = j_1 - m_1, \\ \beta_2 = j_2 + m_2, \quad \beta_3 = j_1 + j_2 - j_3. \quad (2.2)$$

$$\alpha_1 = j_1 - j_3 + m_2, \quad \alpha_2 = j_2 - j_3 - m_1, \quad (2.3)$$

$$\Delta(xyz) = [(x+y-z)! (x-y+z)! (-x+y+z)! / \\ (x+y+z+1)!]^{1/2}, \quad (2.4)$$

and

$$[\lambda] = (2\lambda + 1)^{1/2}. \quad (2.5)$$

The function $\Delta(xyz)$ vanishes unless the usual triangular condition amongst any three angular momenta: $|x - y| \leq z \leq x + y$, is satisfied.

In ref. 4, we have shown that there exist a set of six ${}_3F_2(1)$ s for the 3- j coefficient, which is necessary and sufficient to account for the known 72 symmetries of this coefficient. In place of (2.1) we then have:

$$C(j_1 j_2 j_3; m_1 m_2 - m_3) = \delta(m_1 + m_2, m_3)$$

$$(-1)^{j_1 - j_2 - m_3} [j_3] \begin{bmatrix} j_1 & j_2 & j_3 \\ m_1 & m_2 & m_3 \end{bmatrix}, \quad (2.6)$$

$$\begin{bmatrix} j_1 & j_2 & j_3 \\ m_1 & m_2 & m_3 \end{bmatrix} = \left\| \begin{array}{ccc} -j_1 + j_2 + j_3 & j_1 - j_2 + j_3 & j_1 + j_2 - j_3 \\ j_1 - m_1 & j_2 - m_2 & j_3 - m_3 \\ j_1 + m_1 & j_2 + m_2 & j_3 + m_3 \end{array} \right\| \\ = \|R_{ik}\|, \quad (2.7)$$

$$\|R_{ik}\| = \delta(m_1 + m_2 + m_3) \left[\prod_{i,k=1}^3 R_{ik}! / (J+1)! \right]^{1/2}$$

$$(-1)^{\theta(pqr)} [\Gamma(1-A, 1-B, 1-C, D, E)]^{-1} \\ {}_3F_2(ABC; DE; 1), \quad (2.8)$$

where

$$A = -R_{2p}, \quad B = -R_{3q}, \quad C = -R_{1r}, \quad D = 1 + R_{3r} - R_{2p},$$

$$E = 1 + R_{2r} - R_{3q}, \quad \Gamma(x, y, \dots) = \Gamma(x) \Gamma(y) \dots,$$

$$J = j_1 + j_2 + j_3, \quad (2.9)$$

$$\theta(pqr) = R_{3p} - R_{2q} \text{ for even } (pqr) \text{ and } \theta(pqr) = \\ = R_{3p} - R_{2q} + J \text{ for odd } (pqr), \quad (2.10)$$

for all permutations of $(pqr) = (123)$.

In ref. 5, Wills has arranged the sum in (2.1) into a nested form:

$$S(j_1 j_2 j_3; m_1 m_2 m_3) = \frac{(-1)^\lambda}{a_0! b_0!} \left[1 - \frac{a_0}{b_1} \left(1 - \frac{a_1}{b_2} \left(1 - \dots \right) \right) \right], \quad (2.11)$$

where

$$a_i = \prod_{l=1}^3 (\beta_l - \lambda - i), \quad b_i = (\lambda + i) \prod_{k=1}^2 (\lambda + i - \alpha_k), \quad (2.12)$$

with

$$\lambda = t_{\min}, \quad N = t_{\max} - t_{\min} + 1 \text{ and } 0 \leq i \leq (N-1). \quad (2.13)$$

In all the Clebsch-Gordan function programs we include the following special formulae:

$$C(j_1 j_2 j_3; m_1 m_2 m_3) = 0, \text{ if } m_1 = m_2 = 0 \text{ and } J \text{ is odd} \\ = 1, \text{ if } j_1 = 0 \text{ or } j_2 = 0 \\ = (-1)^{j_1 - m_1} / [j_1], \text{ if } j_3 = 0 \\ = \frac{(-1)^{J_1 - J_2 + J/2} [j_3] \Delta(j_1 j_2 j_3) (J/2)!}{(J/2 - j_1)! (J/2 - j_2)! (J/2 - j_3)!}, \text{ if } m_1 = m_2 = 0. \quad (2.14)$$

The conventional expression for the Racah coefficient [1] is given by:

$$W(abcd; ef) = N \sum_p (-1)^p (p+1)!$$

$$\left[\prod_{i=1}^4 (p - \alpha_i)! \prod_{j=1}^3 (\beta_j - p)! \right]^{-1}, \quad (2.15)$$

where the range of p is:

$$p_{\min} \leq p \leq p_{\max}, \quad (2.16)$$

with

$$\begin{aligned} p_{\min} &= \max(\alpha_1, \alpha_2, \alpha_3, \alpha_4), \quad p_{\max} = \min(\beta_1, \beta_2, \beta_3), \\ \alpha_1 &= a + b + e, \quad \alpha_2 = c + d + e, \quad \alpha_3 = a + c + f, \\ \alpha_4 &= b + d + f, \quad \beta_1 = a + b + c + d, \quad \beta_2 = a + d + e + f, \\ \beta_3 &= b + c + e + f, \end{aligned} \quad (2.17)$$

and

$$N = (-1)^{a+b+c+d} \Delta(abe) \Delta(cde) \Delta(acf) \Delta(bdf) \quad (2.18)$$

In ref. 6, we have shown that when (2.15) is rewritten in terms of generalized hypergeometric functions of unit argument, we get:

$$\begin{aligned} W(abcd; ef) &= (-1)^{E+1} N \Gamma(1 - E) \\ &\times [\Gamma(1 - A) \Gamma(1 - B) \Gamma(1 - C) \Gamma(1 - D) \\ &\times \Gamma(F) \Gamma(G)]^{-1} {}_4F_3(ABCD; EFG; 1), \end{aligned} \quad (2.19)$$

where the following are the numerator and denominator parameters belonging to the set I of three ${}_4F_3(1)$ s:

$$\begin{aligned} A &= e - a - b, \quad B = e - c - d, \quad C = f - a - c, \quad D = f - b - d, \\ E &= -a - b - c - d - 1, \quad F = e + f - a - d + 1, \quad G = e + f - b - c + 1, \end{aligned} \quad (2.20)$$

$$\begin{aligned} A &= d - b - f, \quad B = a - b - e, \quad C = d - c - e, \quad D = a - c - f, \\ E &= -b - c - e - f - 1, \quad F = a + d - e - f + 1, \\ G &= a + d - b - c + 1, \end{aligned} \quad (2.21)$$

$$\begin{aligned} A &= b - a - e, \quad B = b - d - f, \quad C = c - a - f, \quad D = c - d - e, \\ E &= -a - d - e - f - 1, \quad F = b + c - e - f + 1, \\ G &= b + c - a - d + 1. \end{aligned} \quad (2.22)$$

By adopting a procedure similar to that in ref. 6, we have obtained in ref. 7, the expression:

$$\begin{aligned} W(abcd; ef) &= (-1)^{A-2} N \Gamma(A) [\Gamma(1 - B) \Gamma(1 - C) \\ &\times \Gamma(1 - D) \Gamma(E) \Gamma(F) \Gamma(G)]^{-1} \\ &\times {}_4F_3(ABCD; EFG; 1), \end{aligned} \quad (2.23)$$

where the following are numerator and denominator parameters belonging to the set II of four ${}_4F_3(1)$ s:

$$\begin{aligned} A &= a + b + e + 2, \quad B = a - c - f, \quad C = b - d - f, \quad D = e - c - d, \\ E &= a + b - c - d + 1, \quad F = a + e - d - f + 1, \\ G &= b + e - c - f + 1, \end{aligned} \quad (2.24)$$

$$\begin{aligned} A &= c + d + e + 2, \quad B = c - a - f, \quad C = d - b - f, \quad D = e - a - b, \\ E &= c + d - a - b + 1, \quad F = c + e - b - f + 1, \\ G &= d + e - a - f + 1, \end{aligned} \quad (2.25)$$

$$\begin{aligned} A &= a + c + f + 2, \quad B = c - d - e, \quad C = a - b - e, \quad D = f - b - d, \\ E &= a + c - b - d + 1, \quad F = a + f - d - e + 1, \\ G &= c + f - b - e + 1, \end{aligned} \quad (2.26)$$

$$\begin{aligned} A &= b + d + f + 2, \quad B = b - a - e, \quad C = d - c - e, \quad D = f - a - c, \\ E &= b + d - a - c + 1, \quad F = b + f - c - e + 1, \\ G &= d + f - a - e + 1. \end{aligned} \quad (2.27)$$

The ${}_pF_q(1)$ s in expressions (2.8), (2.19) and (2.23) have negative numerator parameters, so that they represent terminating series and the ${}_4F_3(1)$ s are also of the Saalschutzyan type [11]. Furthermore, the ${}_pF_q(1)$ s can be computed numerically using Horner's rule [10] for polynomial evaluation, as:

$$\begin{aligned} {}_pF_q(\alpha_1, \alpha_2, \dots, \alpha_p; \beta_1, \beta_2, \dots, \beta_q; z) \\ = \left[1 + \frac{x_0}{y_0} \left(z + \frac{x_1}{y_1} \left(z + \frac{x_2}{y_2} \left(z + \dots \right) \right) \right) \right], \end{aligned} \quad (2.28)$$

where

$$x_i = \prod_{j=1}^p (\alpha_j + i) \quad \text{and} \quad y_i = (i + 1) \prod_{k=1}^q (\beta_k + i). \quad (2.29)$$

Wills [5] has pointed out that as in the case of the Clebsch–Gordan coefficient, the series expansion in (2.16) can also be cast into a nested form, given by Bretz:

$$S(abcd; ef) = (-1)^\lambda \frac{(c_0)!}{(a_0)!(b_0)!} \left[1 - c_1 \frac{a_0}{b_1} \left(1 - c_2 \frac{a_1}{b_2} \left(1 - \dots \right) \right] \right], \quad (2.30)$$

where

$$a_k = \prod_{j=1}^3 (\beta_j - \lambda - k), \quad b_k = \prod_{i=1}^4 (\lambda + k - \alpha_i),$$

$$c_k = (\lambda + k + 1), \quad (2.31)$$

with

$$\lambda = p_{\min}, N = p_{\max} - p_{\min} + 1 \text{ and } 0 \leq k \leq (N - 1). \quad (2.32)$$

In all the Racah coefficient function programs we include the following special formulae:

$$W(abcd; ef) = (-1)^{a+b+c+d} \times (-1)^{b+c+e+f} ([a][d])^{-1}, \quad \text{if } b, c \text{ or } e = 0,$$

$$\times (-1)^{a+d+e+f} ([b][c])^{-1}, \quad \text{if } a, d \text{ or } f = 0. \quad (2.33)$$

We have used the expressions (2.1), (2.8) and (2.11) for calculating the Clebsch–Gordan coefficient and the corresponding function programs are named as C, CF and CW, respectively. The expressions (2.15), (2.19), (2.23) and (2.30) have been used for calculating the Racah coefficient and the corresponding function programs are named as W, WF, W4F and WBZ, respectively.

3. Program structure and subprograms

The complete deck comprises the main program which calls the function subprograms C, CF and CW, for the Clebsch–Gordan coefficient and the function subprograms W, WF, W4F and WBZ, for the Racah coefficient and the time taken for computing the value of an angular momentum coefficient

for a given set of parameters is noted with the help of an interval timer routine implemented on the IBM 370/168 computer.

3.1. Calling and operation of the main program

Communications with the afore-mentioned function subprograms is established through the following statements:

C(AJ1, AJ2, AJ3, AM1, AM2, AM3)
CF(AJ1, AJ2, AJ3, AM1, AM2, AM3)
CW(AJ1, AJ2, AJ3, AM1, AM2, AM3)
W(A, B, C, D, E, F)
WF(A, B, C, D, E, F)
W4F(A, B, C, D, E, F)
WBZ(A, B, C, D, E, F)

where the arguments of the first three function subprograms stand for the parameters $j_1, j_2, j_3, m_1, m_2, m_3$ and the arguments of the last four function subprograms stand for the parameters a, b, c, d, e and f .

At the very beginning the logarithms of the first 500 factorials are calculated and set up as an array in a dimensioned COMMON block, FCT(500). This is the usual approach, used by Tamura [3], and has two advantages: the array is available for a look-up whenever needed, which is much faster than forming each factorial separately and the overflow due to products of large factorials is controlled, since $\ln(n!)$ is much smaller than $n!$ and multiplication of factorials is replaced by first adding their logarithms and then exponentiating the sum. However, as has been aptly pointed out by Wills [5], the straight-forward method of Tamura [3] for programming the Clebsch–Gordan and Racah coefficients, given by (2.1) and (2.15), respectively, necessitates the exponentiation of each term in the summation, which reintroduces the possibility of overflow, in single-precision computation, besides being time consuming, especially for large values of angular momenta where the number of terms to be summed can be large. Replacing the sum in (2.1) by the nested form (2.11), Wills [5] removed the factorials from the sum, thereby not only avoiding the possibility of overflow but also making the computation faster. A similar modification referred to by Wills [5], has been worked out by Bretz [8] for the Racah coefficient.

Our approach is to utilize the sets of generalized hypergeometric functions of unit argument for the

computation of the Clebsch–Gordan and Racah coefficients. The main advantages of this approach are:

- (i) the adoption of Horner's rule [10] for evaluating the ${}_pF_q(1)$, given by (2.28), places our approach on the same footing as the Wills method and in the nested form the number of multiplications is a minimum—in fact, the expressions of Bretz (2.30) to (2.32) can be shown to be identical to the set II of ${}_4F_3(1)$ s given by (2.23) to (2.27), and
- (ii) the ${}_pF_q(z)$ has the property that if anyone of the numerator parameters is zero, which corresponds here to either one of the angular momenta having a stretched value (e.g.: $j_3 = j_1 + j_2$) or to some special values of projections in the case of the Clebsch–Gordan coefficient (i.e.: $m_i = \pm j_i$, $i = 1, 2, 3$), then its value is 1 and the sum in (2.1) and (2.15) reduces to a single term. This condition implies that in our formulation 18 special formulae for the Clebsch–Gordan coefficient and 12 special formulae for the Racah coefficient are incorporated.

After setting up the array for the factorials, the values of j_1, j_2, j_3, m_1, m_2 and m_3 are read. An interval timer routine implemented on the IBM 370/168 installation at the University of Bonn, is called to note the time. Then the Clebsch–Gordan coefficient function subprogram C is computed n times in a DO loop. After this, the interval timer routine is called again and the difference in the successive times noted divided by n gives the average time taken by C to compute a single Clebsch–Gordan coefficient. The parameters, the value of the coefficient and its average execution time are printed before repeating the same procedure with the function subprograms CF and CW. If t_1, t_2 and t_3 are the execution times for computing the Clebsch–Gordan coefficient using the conventional series (2.1), the set of ${}_3F_2(1)$ s (2.8) and the formula (2.11) of Wills, then t_1/t_2 and t_1/t_3 will give the advantage factors due, respectively, to the use of our approach and that of Wills, over the conventional procedure.

After checking the end of the data set for the Clebsch–Gordan computation, control is transferred to read the values of the arguments of the Racah coefficient: a, b, c, d, e and f . An identical procedure to that used above is employed for getting the average execution times for computing the Racah

coefficient using, successively, the function subprograms W, WF, W4F and WBZ. The arguments, the value of the Racah coefficient, the execution time, as well as the advantage factors due to the use of set I of ${}_4F_3(1)$ s, set II of ${}_4F_3(1)$ s and the Wills–Bretz formula, over the conventional procedure are printed.

A given function subprogram is called n times in a loop and the average execution time is noted mainly because of the nature of a modern computer system like the IBM 370/168, which has a hierarchy of memories. The execution time on such a computer, in its multiprogramming and batch processing mode, is a minimum, if the information being processed is in the High Speed Buffer storage. The probability of retaining the information in the High Speed Buffer storage is increased when the execution is in a loop. The fluctuations in computer timing are considerably reduced by computing a given function subprogram n times in a loop after calling it, where n was arbitrarily set to be 20, 50 and 100. This procedure yields fairly constant values for the average execution time for a given function subprogram, enabling us to make a reliable statement about the comparative execution times of function subprograms, in the normal user's mode.

Also, to get precise timings for ten of the coefficients, with large values of angular momenta, computed by Bretz [8], another main program was employed, which was run with all the function subprograms, in a special run, with the computer not being in its usual multiprogramming and batch processing mode.

3.2. Subprograms

The following is a brief resume of the various function subprograms:

- (1) Function C employs the single sum series (2.1) for the Clebsch–Gordan coefficient. Here, as well as in the cases of CF and CW, the triangular inequality is first checked, with the help of a function subprogram TRIA and then the special conditions — anyone of the angular momenta being zero or all the projections being zero — are sorted out. The range of the summation index t , given by (2.2) is found and then the expression (2.1) is computed in a loop for t , with each term being given as the exponent of a sum of

logarithms. The value of the Clebsch–Gordan coefficient is returned as C.

(2) Function CF utilizes the set of ${}_3F_2(1)$ s given by (2.8)–(2.10). Since anyone of the given ${}_3F_2(1)$ s is defined only when its denominator parameters are positive and satisfy the conditions: $R_{3r} \geq R_{2p}$ and $R_{2r} \geq R_{3q}$, a check of these conditions enables the selection of the valid ${}_3F_2(1)$ parameter set. Then we check whether any one of the numerator parameters is zero, and if so set ${}_3F_2(1) = 1$ and skip the program segment for computing the ${}_3F_2(1)$. Otherwise, the number of terms in the series is found and the ${}_3F_2(1)$ calculated using the Horner's rule. In this case, as well as in the case of CW, the exponentiation of the logarithmic sum is performed outside of the summation loop only once. The value of the Clebsch–Gordan coefficient is returned as CF.

(3) Function CW uses the special formula given by Wills (2.11) for the sum in (2.1). The number of terms in the series is found and the nested expression computed and the value of the Clebsch–Gordan coefficient returned as CW.

(4) Function W is for the single sum series (2.15) for the Racah coefficient. Here, as well as in the cases of WF, W4F and WBZ, the four triangular inequalities are checked using TRIA and then the cases when any one of the arguments being zero are sorted out. The range of the summation index p , given by (2.16), is found, the summation is carried out with each term being computed as the exponent of a sum of logarithms and the value of the Racah coefficient returned as W.

(5) Function WF is for the set I of ${}_4F_3(1)$ s, given by (2.19), for the Racah coefficient. Following the arguments in ref. 6, we check for the two denominator parameters being positive and accordingly select the parameter set (2.20), (2.21) or (2.22) for the ${}_4F_3(1)$. We check whether any of the numerator parameters is zero, and if so set the ${}_4F_3(1) = 1$. Otherwise, the number of terms in the series is found and the ${}_4F_3(1)$ calculated using the Horner's rule and the value of the Racah coefficient is returned as WF. In this subprogram, as well as in W4F and WBZ, there is only one exponentiation of the logarithmic sum in the last step.

(6) Function W4F is similar in all respects to function WF except for the fact that W4F is for the set II of ${}_4F_3(1)$ s given by (2.23) and here we

have to check for all the three denominator parameters being positive, as pointed out in ref. 7, to select one of the four sets of parameters given by (2.24)–(2.27).

(7) Function WBZ is for the Bretz formula (2.30) for the sum in (2.15). The number of terms in the series is found and the nested expression computed and the value of the Racah coefficient returned as WBZ.

All the seven function programs refer to the array FCT(500) for factorials placed in the Common block and utilize the following function subprograms:

(8) Function PHASE(N) is for finding $(-1)^N$.

(9) Function TRIA(x, y, z) checks for the triangular inequality $|x - y| \leq z \leq x + y$, which must be satisfied by any three angular momenta belonging to a triad.

4. Results and discussion

A comparison of our method with the methods of Tamura [3], Wills [5] and Bretz [8] was made on the IBM 370/Model 168 computer at the University of Bonn. The main program was run for more than 100 values of the Clebsch–Gordan coefficient and for more than 100 values of the Racah coefficient, several times and the results scrutinized carefully. The execution time to compute a Clebsch–Gordan coefficient or a Racah coefficient is found to depend on the magnitude of the arguments.

For small values of angular momenta, the three Clebsch–Gordan coefficient function subprograms: C, CF and CW and the four Racah coefficient function subprograms: W, WF, W4F and WBZ yielded numerical results to the same degree of accuracy. However, even for small arguments, the present function subprograms: CF for the Clebsch–Gordan

Table 1
C(j 30 40; 2 2 4)

j	N	C	CF	CW	E.P.
10	1	0.42496	0.42496	0.42496	0.42488
15	6	0.17302	0.17178	0.17178	0.17173
20	11	-0.20563	-0.16843	-0.16843	-0.16835
25	16	-0.18888	-0.06166	-0.06166	-0.06157
30	21	0.40885	0.16726	0.16726	0.16746

Table 2
 $W(35\ 35\ 40\ 40; 26\ j)$

j	N	W	WF	W4F	WBZ	E.P.
15	11	-0.003277	-0.003290	-0.003293	-0.003293	-0.003290
25	21	0.000430	0.000340	0.000340	0.000340	0.000340
33	27	-0.000197	0.000374	0.000375	0.000375	0.000375
35	27	0.001427	0.002295	0.002309	0.002309	0.002301
40	27	0.000955	0.000701	0.000704	0.000704	0.000698
45	27	-0.001878	-0.001782	-0.001779	-0.001779	-0.001787

coefficient and WF and W4F for the Racah coefficient, have been almost always faster, by at best 20%, when compared with the conventional programs of Tamura and the improved programs of Wills-Bretz, in the usual user's mode of computer operation.

For large values of angular momenta, the Clebsch-Gordan and Racah coefficient function subprograms yield identical numerical results only when they are run on Extended Precision (E.P.). In single precision, the method of Tamura fails, as pointed out by Wills [5]. Tables 1 and 2 give the values of $C(j\ 30\ 40; 2\ 2\ 4)$ and $W(35\ 35\ 40\ 40; 26\ j)$, respectively, where the first column gives the j value, the second column gives the number of terms summed and the last column gives the results computed in Extended Precision, while the intermediate columns, labelled by the names of the function subprograms, give the values of the angular momentum coefficients obtained in single precision. From table 1 we note that our function subprogram CF and that of Wills, CW, give identical numerical results with negligibly small round-off errors. From table 2 we note that while our function subprogram W4F gives results identical to Bretz's WBZ, our function subprogram WF gives numerical results which are agreeable though slightly different.

The timings in the usual user's mode of computer operation show:

- (i) an improvement factor of 2 to 3 for the function subprogram CW of Wills, over C, for $j \geq 18$, and our function subprogram CF is even faster by $\leq 20\%$;
- (ii) an improvement factor of 1.5 to 2 for the function subprogram WBZ of Bretz, over W, for $j \geq 15$, while our function subprograms WF and W4F are even faster by $\leq 15\%$.

However, the main program was subjected to a

special run (mentioned earlier) to clock the precise timings for the various function subprograms and we obtained the following results:

(i) For the ten Clebsch-Gordan coefficients $C(j\ 30\ 40; 2\ 2\ 4)$, with $41 \leq j \leq 50$, the function subprograms C, CF and CW took, respectively, 171.3 μ s, 57.4 μ s and 60.6 μ s, establishing thereby advantage factors of ≈ 3 and 2.83 for CF and CW over C. In other words, the present function subprogram CF is faster by about 5% than that of Wills.

(ii) For the ten Racah coefficients $W(35\ 35\ 40\ 40; 26\ j)$, with $35 \leq j \leq 44$, the function subprograms W, WF, W4F and WBZ took, respectively, 165.4 μ s, 78 μ s, 79.14 μ s and 87.46 μ s, establishing thereby advantage factors of 2.12, 2.09 and 1.89 for WF, W4F and WBZ over W. In other words, the present function subprograms WF and W4F are faster by 10.8% and 9.5%, respectively, than that of Bretz's WBZ.

In this case also, we did average the times for five runs at a time and there is at best a 1% fluctuation in the computer time, in this special mode of computer operation.

The $ls-jj$ or $9j$ coefficient [2] calculated in terms of the Racah coefficients has also been found to be calculable 5 to 10% faster with the function subprograms WF and W4F than with WBZ, especially for large values of angular momenta.

In conclusion, when large angular momenta are involved, one should resort to the function subprograms CF or CW for the Clebsch-Gordan coefficient and to the function subprograms WF, W4F and WBZ for the Racah coefficient, since these, besides being better than the conventional programs of Tamura in single-precision, are 2-3 times faster even

in extended precision, when all the function subprograms yield identical numerical results. The sets of ${}_pF_q(1)$ s which have been shown to be necessary and sufficient to account for the symmetries of the Clebsch–Gordan and Racah coefficients are found to be not only useful for numerically computing the same but also have a small but perceptible advantage over the best available programs, since they are somewhat faster at best by 5–15%.

Acknowledgements

One of us (K.S.R.) wishes to thank the Humboldt Foundation for the award of a Fellowship; Prof. K. Bleuler for his kind hospitality at the Institut für Theoretische Kernphysik; Mr. V. Aravamudhan and Mr. T.K. Basu of the IBM 370/155 Computer Centre at I.I.T., Madras, where the initial computation was performed, for several discussions; Mr. Hans Reiger of the IBM 370/168 Computer Centre at the University of Bonn for his interest in the problem and for the special runs of the program; Dr. S.C.K. Nair and

Dr. S.K. Sharma for fruitful discussions on some aspects of the problem.

References

- [1] G. Racah, *Phys. Rev.* 62 (1942) 438.
- [2] A. de Shalit and I. Talmi, *Nuclear Shell Theory*, Academic Press, New York, London, 1963).
- [3] T. Tamura, *Comput. Phys. Commun.* 1 (1970) 337.
- [4] K. Srinivasa Rao, *J. Phys. A* 11 (1978) L69; K. Venkatesh, *J. Math. Phys.*, to be published.
- [5] J.G. Wills, *Comput. Phys. Commun.* 2 (1971) 381.
- [6] K. Srinivasa Rao, T.S. Santhanam and K. Venkatesh, *J. Math. Phys.* 16 (1975) 1528.
- [7] K. Srinivasa Rao and K. Venkatesh, *Proc. 5th Int. Coll. on Gp. Theor. Methods in Phys.*, Montreal (1976).
- [8] V. Bretz, *Acta Phys. Acad. Sci. Hungaricae* 40 (1976) 255.
- [9] *Quantum Theory of Angular Momentum*, L.C. Biedenharn and H. Van Dam, eds. (Academic Press, New York, London, 1965); Also, Ya.A. Smorodinski and L.A. Shelepin, *Sov. Phys. Usp.* 15 (1972) 1.
- [10] John A.N. Lee, *Numerical Analysis for Computers* (Reinhold Pub. Corp., 1966).
- [11] L.J. Slater, *Generalized Hypergeometric Functions* (Cambridge U.P., Cambridge, 1965) ch. 2.